



UNIVERSIDADE FEDERAL DO AMAPÁ
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Disciplina: Inteligência Artificial
Professor: Dr. Adolfo Colares

Daniela Marques Haber Sepeda
Geovanni Rodrigues da Silva
Maria Fernanda Fernandes de Souza Gonçalves
Miguel Peres Lobo

Relatório Técnico

Identificador de Gargalos e Recomendador de Upgrades de Hardware

Macapá
2026

SUMÁRIO

1. PROBLEMA E MOTIVAÇÃO..... 3

2. DADOS.....4

3. METODOLOGIA..... 5

4. RESULTADOS..... 5

5. ÉTICA E LIMITAÇÕES..... 6

6. CONCLUSÃO E PRÓXIMOS PASSOS..... 7

1. PROBLEMA E MOTIVAÇÃO

Lentidão e quedas de sistema (downtime) são um problema tanto para grandes empresas quanto para quem gerencia a infraestrutura de laboratórios e PMEs. Quando um servidor ou estação de trabalho para, o prejuízo financeiro e a perda de produtividade são imediatos. O grande problema é que na maioria das manutenções é encontrado dificuldade de processo, o que leva à compra de peças erradas (como colocar mais RAM quando o gargalo real era a velocidade de leitura do disco).

Foi escolhido usar Inteligência Artificial (IA) em vez de um sistema simples de regras (if/else) porque gargalos dependem de contexto. Se tentássemos usar if/else, esbarraríamos no fato de que 95% de uso de disco pode significar coisas diferentes: em um banco de dados, pode ser falta de capacidade do HD; mas se a RAM também estiver lotada, o disco está apenas compensando a falta de memória (paginação). O nosso modelo de Raciocínio Baseado em Casos (RBC) resolve isso de forma inteligente, comparando o cenário atual com o histórico para entender o contexto inteiro antes de dar o diagnóstico, agindo como um técnico experiente.

Figura 1: Comparativo do custo médio de inatividade (downtime) por hora entre PMEs e grandes corporações.

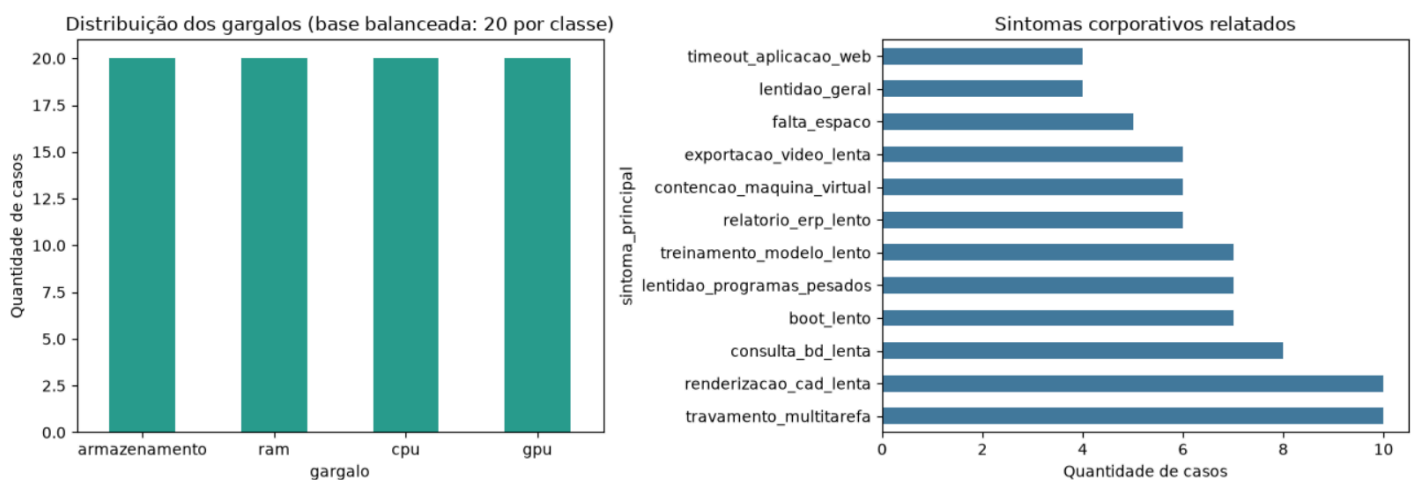


2. DADOS

Como não existem bancos de dados públicos já classificados com "gargalo de hardware e solução", nós mesmos construímos a nossa base.

- Tamanho e fonte: Nossa base final tem 80 casos focados no ambiente corporativo (servidores, ofimática e estações de trabalho).
- O que a EDA (Análise Exploratória) revelou: Na nossa segunda entrega, notamos um desbalanceamento ruim (tínhamos muitos casos de armazenamento e poucos de CPU). Corrigimos isso agora: a base tem exatamente 20 casos para cada um dos 4 gargalos (CPU, RAM, GPU e Armazenamento). Também percebemos que era preciso colocar algumas "pegadinhas" (ambiguidades) nos sintomas textuais para forçar a IA a olhar para os números de uso dos componentes.
- Tratamento: Não tínhamos dados faltantes (nulos), mas precisávamos transformar textos em números. Usamos One-Hot Encoding para as categorias (tipo de máquina, sintoma) e demos pesos sequenciais para peças (ex: HDD vale 0, SSD SATA vale 1, SSD NVMe vale 2) para o algoritmo entender que um é mais rápido que o outro.

Figura 2: Distribuição perfeitamente balanceada das classes de gargalo (esquerda) e frequência dos sintomas corporativos na base (direita).



3. METODOLOGIA

Para fazer a IA funcionar, usamos a técnica de Raciocínio Baseado em Casos (os 4 Rs) e a biblioteca scikit-learn em Python.

Técnica escolhida: O motor do nosso sistema é o algoritmo KNN (K-Nearest Neighbors) usando distância euclidiana com $k=3$. Escolhemos ele porque o RBC precisa "recuperar" os casos mais parecidos do passado (Retrieve) e o KNN faz exatamente isso: acha os 3 vizinhos matematicamente mais próximos do problema atual. O diagnóstico final é feito por votação (Reuse) entre esses 3 vizinhos.

Evitando Overfitting: Como 80 casos é um número pequeno, não podíamos simplesmente cortar a base no meio (treino e teste). Usamos a validação Leave-One-Out. Ela treina o modelo com 79 casos e testa no que sobrou, repetindo isso 80 vezes.

Tratamento de escala: Usamos o MinMaxScaler para deixar todas as variáveis entre 0 e 1. Isso garante que um número grande (como 32GB de RAM) não ofusque um número pequeno (como 100% de uso de CPU). Colocamos isso dentro de um Pipeline para garantir que a escala fosse calculada só nos dados de treino, evitando o vazamento de dados (data leakage).

4. RESULTADOS

Nossa métrica principal para avaliar o modelo foi a Acurácia Global, que é a mais indicada aqui já que nossas classes estão perfeitamente balanceadas (20 casos cada).

- Métricas: O modelo alcançou 80,0% de acurácia geral.
- Matriz de Confusão: Olhando para os acertos e erros, notamos que a IA acerta 100% das vezes quando o problema é Placa de Vídeo (GPU), porque cargas de trabalho pesadas como renderização são bem exclusivas. A maior parte das confusões do modelo acontece na "tríade" CPU-RAM-Disco, porque na vida real esses componentes afetam uns aos outros.
- Comparação com e sem Retenção: Testamos a capacidade da IA de aprender coisas novas (a etapa Retain). Começamos com uma base pequena (24 casos). Sem a capacidade de aprender, a IA acertava só 67,9%. Quando ativamos a retenção (ela foi guardando os casos que resolvia), a precisão subiu para 85,7% nos últimos testes. Ou seja, provamos que o sistema fica mais inteligente com o tempo.

5. ÉTICA E LIMITAÇÕES

Trazendo as discussões da Aula 08 para o nosso projeto, precisamos ser transparentes sobre os vieses e limites técnicos:

- **Viés nos dados:** Nós construímos e rotulamos os dados. Isso significa que a IA carrega o nosso viés técnico. Se a equipe tem a tendência de achar que "lentidão geral" é sempre culpa da CPU, o modelo vai aprender e repetir esse comportamento, o que pode fazer o cliente comprar a peça errada.
- **Privacidade (LGPD):** Se este sistema for implementado na rede de uma empresa real, ele deverá extrair apenas a telemetria do hardware (uso de processador, temperatura). É estritamente proibido coletar nomes de usuários logados, IPs ou arquivos abertos, para não ferir as leis de proteção de dados.
- **O que o modelo NÃO consegue fazer:** A IA é ótima para diagnosticar lentidão contínua e sobrecarga (gargalos). Porém, ela é completamente incapaz de prever falhas súbitas. Se a placa-mãe queimar por um curto-circuito ou o servidor desligar por falta de energia, a IA não vai prever isso, pois esses eventos físicos não geram lentidão prévia no Windows/Linux para o KNN comparar.

O que o modelo prevê (Gargalos)	O que o modelo NÃO prevê (Falhas Críticas)
Degradação de Desempenho: Lentidão progressiva em sistemas operacionais.	Falhas Elétricas: Queima de componentes por surtos de tensão ou curtos-circuitos.
Saturação de Recursos: Gargalos de CPU, RAM, GPU ou Armazenamento (I/O).	Danos Físicos: Quebra mecânica de conectores, portas USB ou pinos de CPU.
Estrangulamento (Thermal Throttling): Perda de performance por superaquecimento.	Interrupções Súbitas: Quedas de energia, desligamentos por fonte queimada ou problemas na rede elétrica.
Incompatibilidade de Perfil: Configuração de hardware insuficiente para o uso atual.	Falhas de Fabricação: Defeitos de fábrica que não apresentam telemetria prévia de degradação.

6. CONCLUSÃO E PRÓXIMOS PASSOS

A nossa abordagem provou que é possível sair da tentativa e erro na manutenção de TI e usar IA para justificar compras de hardware com base em dados históricos. O recurso de retenção mostrou que a ferramenta ganha valor com o uso contínuo.

Se tivéssemos mais um mês de projeto, nossos próximos passos seriam:

1. Melhorar a matemática: Trocar a distância euclidiana simples por uma distância ponderada, dando mais peso para o "sintoma relatado" do que para o "tipo de sistema operacional".
2. Coleta automática: Integrar a biblioteca psutil no Python para que o sistema puxe os dados da máquina automaticamente, sem o usuário precisar digitar o uso de CPU e RAM manualmente.
3. Teste em ambiente real: Aplicar o sistema para rodar ativamente no monitoramento das estações de trabalho e nos servidores do laboratório da universidade, validando a ferramenta com telemetria viva e usuários reais.